

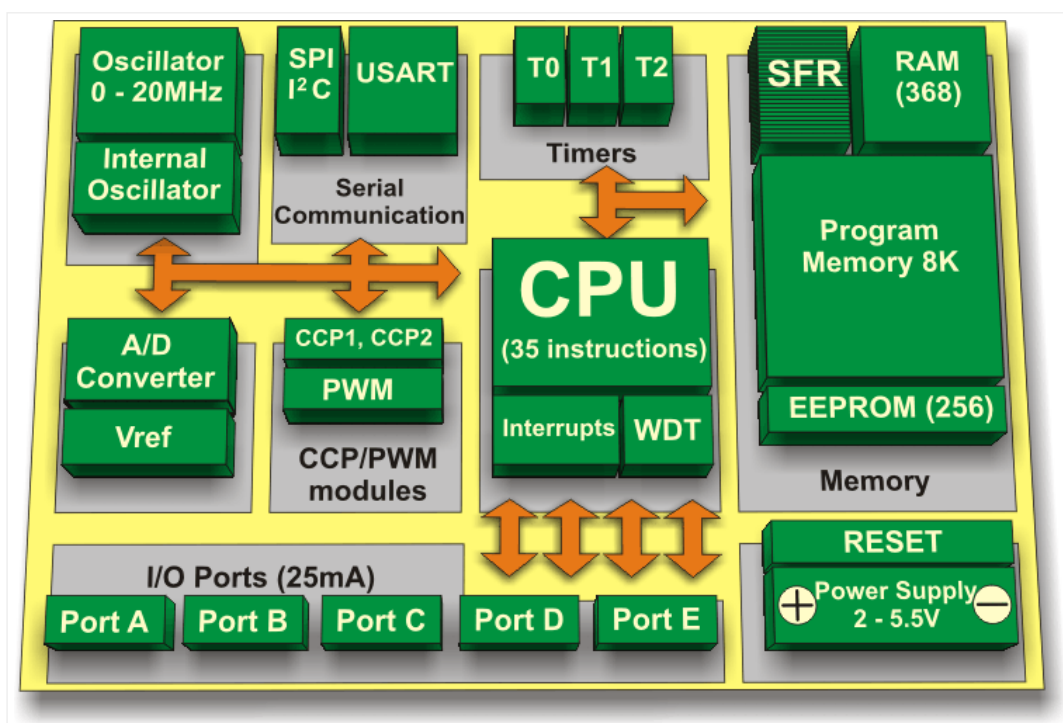
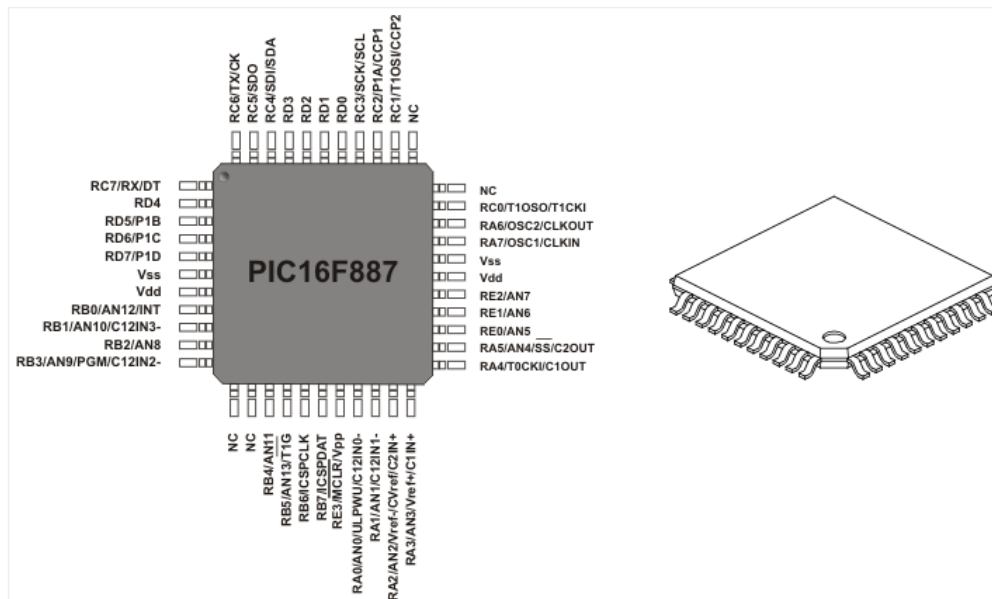
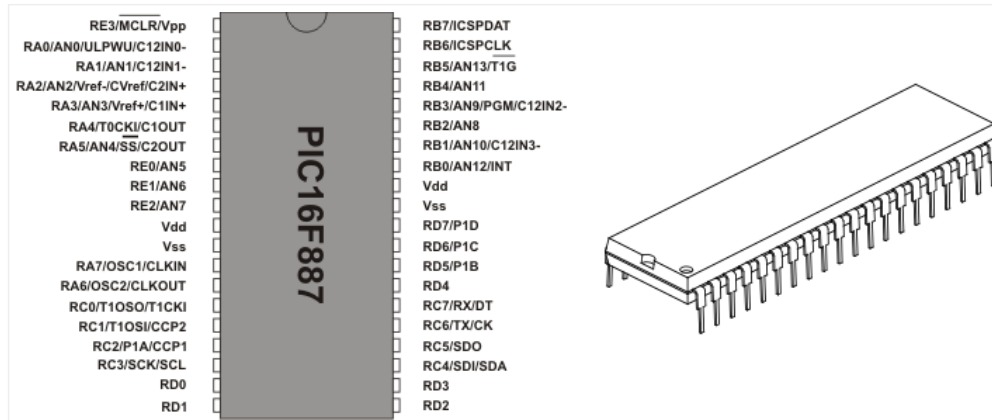


3.1 THE PIC16F887 BASIC FEATURES

MIKROELEKTRONIKA

- **RISC architecture**
 - Only 35 instructions to learn
 - All single-cycle instructions except branches
- **Operating frequency 0-20 MHz**
- **Precision internal oscillator**
 - Factory calibrated
 - Software selectable frequency range of 8MHz to 31KHz
- **Power supply voltage 2.0-5.5V**
 - Consumption: 220uA (2.0V, 4MHz), 11uA (2.0 V, 32 KHz) 50nA (stand-by mode)
- **Power-Saving Sleep Mode**
- **Brown-out Reset (BOR) with software control option**
- **35 input/output pins**
 - High current source/sink for direct LED drive
 - software and individually programmable *pull-up* resistor
 - Interrupt-on-Change pin
- **8K ROM memory in FLASH technology**
 - Chip can be reprogrammed up to 100.000 times
- ***In-Circuit Serial Programming Option***
 - Chip can be programmed even embedded in the target device
- **256 bytes EEPROM memory**
 - Data can be written more than 1.000.000 times
- **368 bytes RAM memory**
- **A/D converter:**
 - 14-channels
 - 10-bit resolution
- **3 independent timers/counters**
- **Watch-dog timer**
- **Analogue comparator module with**
 - Two analogue comparators
 - Fixed voltage reference (0.6V)
 - Programmable on-chip voltage reference
- **PWM output steering control**

- **Enhanced USART module**
 - Supports RS-485, RS-232 and LIN2.0
 - Auto-Baud Detect
- **Master Synchronous Serial Port (MSSP)**
 - supports SPI and I2C mode



PINOUT DESCRIPTION

Most pins of the PIC16F887 microcontroller are multi-functional as seen in figure above. For example, designator RA3/AN3/Vref+/C1IN+ for the fifth pin of the microcontroller indicates that it has the following functions:

- RA3 Port A third digital input/output
- AN3 Third analog input
- Vref+ Positive voltage reference
- C1IN+ Comparator C1 positive input

Such pin functionality is very useful as it makes the microcontroller package more compact without affecting its operation. These various pin functions cannot be used simultaneously, but can be changed at any point during operation.

The following tables refer to the PDIP 40 microcontroller.

Name	Number (DIP 40)	Function	Description
RE3/MCLR/Vpp	1	RE3	General purpose input Port E
		MCLR	Reset pin. Low logic level on this pin resets microcontroller.
		Vpp	Programming voltage
RA0/AN0/ULPWU/C12IN0-	2	RA0	General purpose I/O port A
		AN0	A/D Channel 0 input
		ULPWU	Stand-by mode deactivation input
		C12IN0-	Comparator C1 or C2 negative input
RA1/AN1/C12IN1-	3	RA1	General purpose I/O port A
		AN1	A/D Channel 1
		C12IN1-	Comparator C1 or C2 negative input
RA2/AN2/Vref-/CVref/C2IN+	4	RA2	General purpose I/O port A
		AN2	A/D Channel 2
		Vref-	A/D Negative Voltage Reference input
		CVref	Comparator Voltage Reference Output
		C2IN+	Comparator C2 Positive Input
RA3/AN3/Vref+/C1IN+	5	RA3	General purpose I/O port A
		AN3	A/D Channel 3
		Vref+	A/D Positive Voltage Reference Input
		C1IN+	Comparator C1 Positive Input
RA4/T0CKI/C1OUT	6	RA4	General purpose I/O port A
		T0CKI	Timer T0 Clock Input
		C1OUT	Comparator C1 Output
RA5/AN4/SS/C2OUT	7	RA5	General purpose I/O port A
		AN4	A/D Channel 4
		SS	SPI module Input (<i>Slave Select</i>)
		C2OUT	Comparator C2 Output
RE0/AN5	8	RE0	General purpose I/O port E
		AN5	A/D Channel 5
RE1/AN6	9	RE1	General purpose I/O port E
		AN6	A/D Channel 6
RE2/AN7	10	RE2	General purpose I/O port E
		AN7	A/D Channel 7
Vdd	11	+	Positive supply
Vss	12	-	Ground (GND)

Name	Number (DIP 40)	Function	Description
RA7/OSC1/CLKIN	13	RA7	General purpose I/O port A
		OSC1	Crystal Oscillator Input
		CLKIN	External Clock Input
RA6/OSC2/CLKOUT	14	OSC2	Crystal Oscillator Output
		CLKO	Fosc/4 Output
		RA6	General purpose I/O port A
RC0/T1OSO/T1CKI	15	RC0	General purpose I/O port C
		T1OSO	Timer T1 Oscillator Output
		T1CKI	Timer T1 Clock Input
RC1/T1OSO/T1CKI	16	RC1	General purpose I/O port C
		T1OSI	Timer T1 Oscillator Input
		CCP2	CCP1 and PWM1 module I/O
RC2/P1A/CCP1	17	RC2	General purpose I/O port C
		P1A	PWM Module Output
		CCP1	CCP1 and PWM1 module I/O
RC3/SCK/SCL	18	RC3	General purpose I/O port C
		SCK	MSSP module Clock I/O in SPI mode
		SCL	MSSP module Clock I/O in I ² C mode
RD0	19	RD0	General purpose I/O port D
RD1	20	RD1	General purpose I/O port D
RD2	21	RD2	General purpose I/O port D
RD3	22	RD3	General purpose I/O port D
RC4/SDI/SDA	23	RC4	General purpose I/O port A
		SDI	MSSP module <i>Data</i> input in SPI mode
		SDA	MSSP module <i>Data</i> I/O in I ² C mode
RC5/SDO	24	RC5	General purpose I/O port C
		SDO	MSSP module <i>Data</i> output in SPI mode
RC6/TX/CK	25	RC6	General purpose I/O port C
		TX	USART Asynchronous Output
		CK	USART Synchronous Clock
RC7/RX/DT	26	RC7	General purpose I/O port C
		RX	USART Asynchronous Input
		DT	USART Synchronous Data

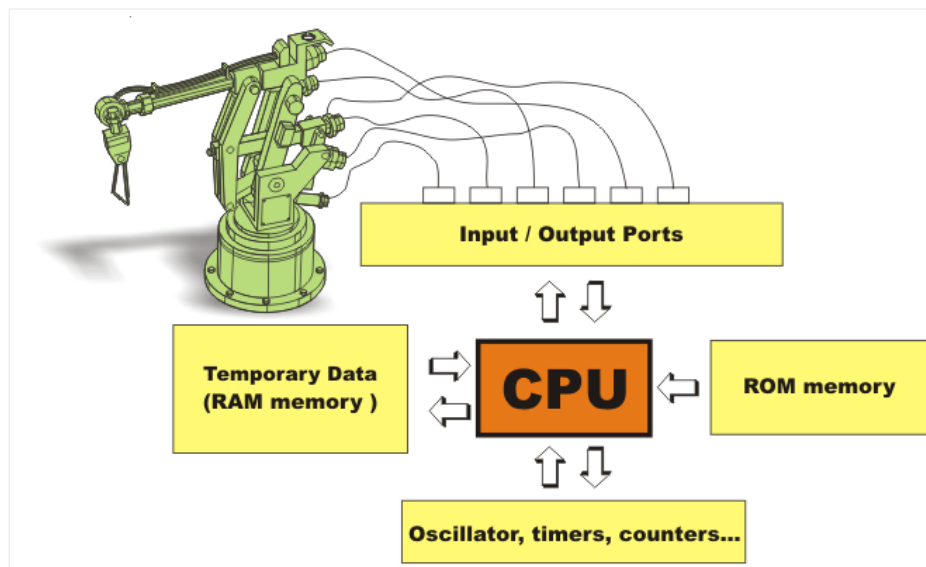
Name	Number (DIP 40)	Function	Description
RD4	27	RD4	General purpose I/O port D
RD5/P1B	28	RD5	General purpose I/O port D
		P1B	PWM Output
RD6/P1C	29	RD6	General purpose I/O port D
		P1C	PWM Output
RD7/P1D	30	RD7	General purpose I/O port D
		P1D	PWM Output
Vss	31	-	Ground (GND)
Vdd	32	+	Positive Supply
RB0/AN12/INT	33	RB0	General purpose I/O port B
		AN12	A/D Channel 12
		INT	External Interrupt
RB1/AN10/C12INT3-	34	RB1	General purpose I/O port B
		AN10	A/D Channel 10
		C12INT3-	Comparator C1 or C2 Negative Input
RB2/AN8	35	RB2	General purpose I/O port B
		AN8	A/D Channel 8
RB3/AN9/PGM/C12IN2-	36	RB3	General purpose I/O port B
		AN9	A/D Channel 9
		PGM	Programming enable pin
		C12IN2-	Comparator C1 or C2 Negative Input
RB4/AN11	37	RB4	General purpose I/O port B
		AN11	A/D Channel 11
RB5/AN13/T1G	38	RB5	General purpose I/O port B
		AN13	A/D Channel 13
		T1G	Timer T1 External Input
RB6/ICSPCLK	39	RB6	General purpose I/O port B
		ICSPCLK	Serial programming Clock
RB7/ICSPDAT	40	RB7	General purpose I/O port B
		ICSPDAT	Programming enable pin

CENTRAL PROCESSOR UNIT (CPU)

We are not going to bore you with the operation of the CPU at this stage. However, we will just state that the CPU is manufactured with RISC technology as it is an important factor when deciding which microcontroller to use.

RISC stands for *Reduced Instruction Set Computer*, which gives the PIC16F877 two great advantages:

- The CPU only recognizes 35 simple instructions. Just to mention that in order to program other microcontrollers in assembly language it is necessary to know more than 200 instructions by heart.
- The execution time is the same for almost all instructions, and lasts for 4 clock cycles. The oscillator frequency is stabilized by a quartz crystal. The execution time of jump and branch instructions is 2 clock cycles. It means that if the microcontroller's operating speed is 20MHz, the execution time of each instruction will be 200nS, i.e. the program will execute 5 million instructions per second!



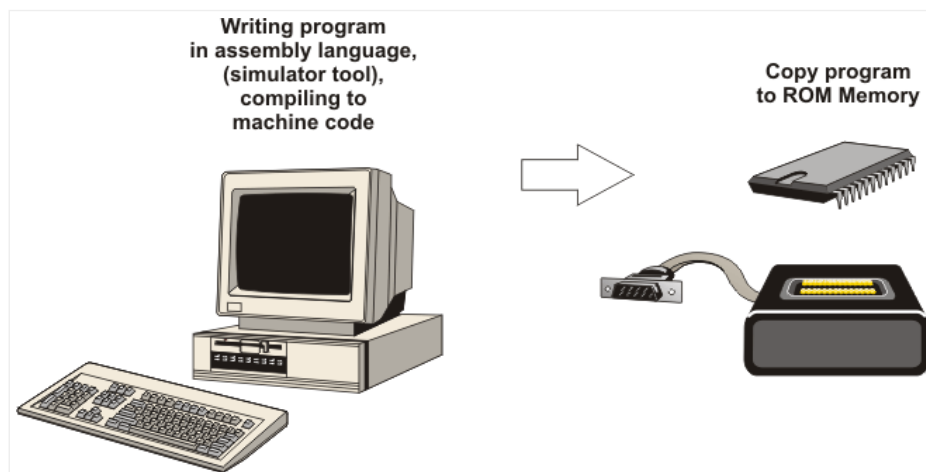
MEMORY

The PIC16F887 has three types of memory ROM, RAM and EEPROM. All of them will be separately discussed since each has specific functions, features and organization.

ROM MEMORY

ROM memory is used to permanently save the program being executed. This is why it is often called 'program memory'. The PIC16F887 has 8Kb of ROM (in total of 8192 locations). Since the ROM memory is made with FLASH technology, its contents can be changed by providing a special programming voltage (13V).

However, it is not necessary to explain it in detail as being automatically performed by means of a special program on the PC and a simple electronic device called the programmer.



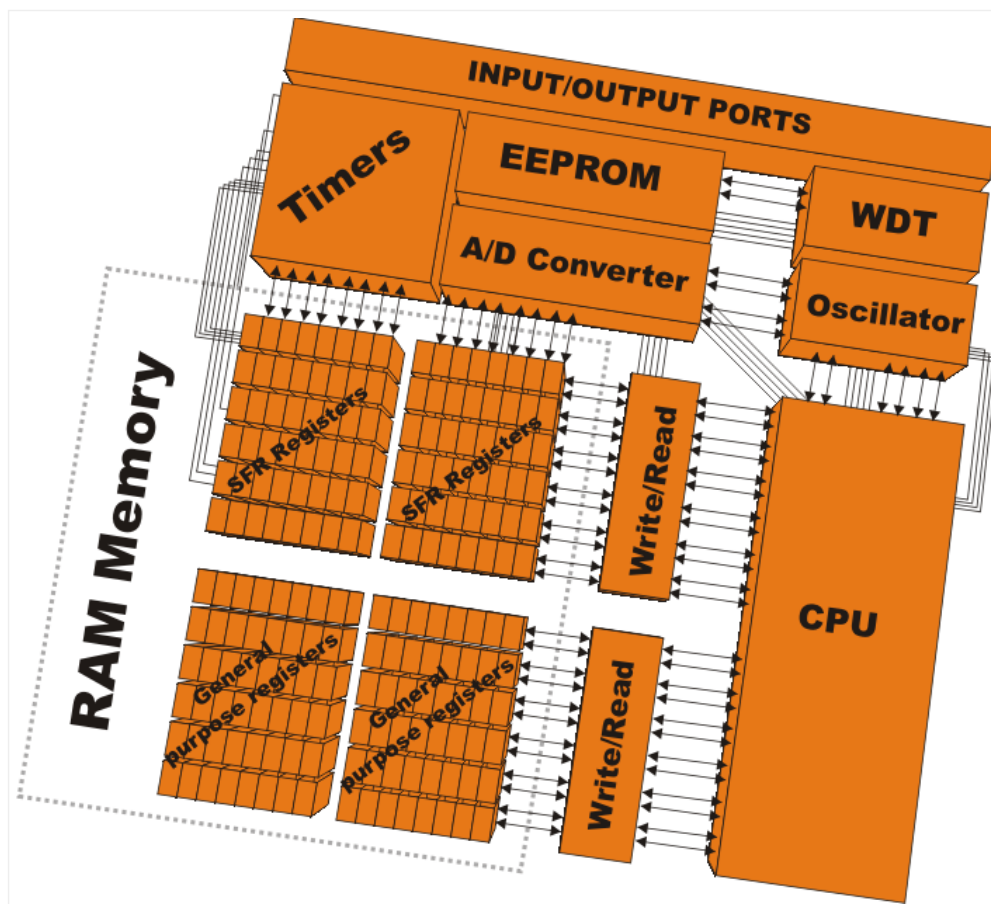
EEPROM MEMORY

Similar to program memory, the contents of EEPROM is permanently saved, even when the power goes off. However, unlike ROM, the contents of EEPROM can be changed during the operation of the microcontroller. This is why this memory (256 locations) is perfect for permanently saving some of the results created and used during the operation.

RAM MEMORY

This is the third and the most complex part of microcontroller memory. In this case, it consists of two parts: general-purpose registers and special-function registers (SFR). All these registers are divided in four memory banks to be explained later in the chapter.

Even though both groups of registers are cleared when power goes off and even though they are manufactured in the same manner and act in a similar way, their functions do not have many things in common.



GENERAL-PURPOSE REGISTERS

General-purpose registers are used for storing temporary data and results created during operation. For example, if the program performs counting (products on the assembly line), it is necessary to have a register which stands for what we in everyday life call 'sum'. Since the microcontroller is not creative at all, it is necessary to specify the address of some general purpose register and assign it that function. A simple program to increment the value of this register by 1, after each product passes through a sensor, should be created.

Now the microcontroller can execute the program as it knows what and where the sum to be incremented is. Similarly, each program variable must be preassigned some of the general- purpose registers.

SPECIAL FUNCTION REGISTERS (SFRS)

Special-function registers are also RAM memory locations, but unlike general-purpose registers, their purpose is predetermined during manufacturing process and cannot be changed. Since their bits are connected to particular circuits on the chip (A/D converter, serial communication module, etc.), any change of their contents directly affects the operation of the microcontroller or some of its circuits. For example, the ADCON0 register controls the operation of A/D converter. By changing its bits it is determined which port pin is to be configured as converter input, the moment conversion is to start as well as the speed of conversion.

Another feature of these memory locations is that they have their names (both registers and their bits), which considerably simplifies the process of writing a program. Since high-level programming languages can use the list of all registers with their exact addresses, it is enough to specify the name of a register in order to read or change its contents.

RAM MEMORY BANKS

Handling banks may be difficult only if you write a program in assembly language. When using higher programming languages such as C and compilers such as **mikroC PRO for PIC**, all you have to do is to specify the register name. On the basis of that, the compiler selects necessary bank and appropriate instructions used for bank selection will be built in the code during the process of compilation. You have been using only assembly language so far and this is the first time you use the C compiler, haven't you? Isn't this a wonderful news?

It may be difficult only if you write a program in assembly language. When using high-level languages such as C and compilers such as **mikroC PRO for PIC**, all you have to do is write the code. On the basis of that, the compiler selects necessary bank and appropriate instructions. Bank selection will be built in the code during the process of compilation. You have been using only assembly language so far and this is the first time you use the C compiler, haven't you? Isn't this a wonderful

Handling banks may be difficult only if you write a program in assembly language. When using higher programming languages such as C and compilers such as **mikroC PRO for PIC**, all you have to do is to specify the register name. On the basis of that, the compiler selects necessary bank and appropriate instructions used for bank selection will be built in the code during the process of compilation. You have been using only assembly language so far and this is the first time you use the C compiler, haven't you? Isn't this a wonderful news?

SFRs bank 0

Address	Name	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
00h	INDF	Indirect register							
01h	TMR0	Timer T0 Register							
02h	PCL	Least Significant Byte of Program Counter							
03h	STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C
04h	FSR	Indirect Data Memory Address Pointer							
05h	PORTA	RA7	RA6	RA5	RA4	RA3	RA2	RA1	RA0
06h	PORTB	RB7	RB6	RB5	RB4	RB3	RB2	RB1	RB0
07h	PORTC	RC7	RC6	RC5	RC4	RC3	RC2	RC1	RC0
08h	PORTD	RD7	RD6	RD5	RD4	RD3	RD2	RD1	RD0
09h	PORTE	-	-	-	-	RE3	RE2	RE1	RE0
0Ah	PCLATH	-	-	-	Upper 5 bits of Program Counter				
0Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF
0Ch	PIR1	-	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF
0Dh	PIR2	OSFIF	C2IF	C1IF	EEIF	BCLIF	ULPWUIF	-	CCP2IF
0Eh	TMR1L	Least Significant Byte of the 16-bit Timer TMR0							
0Fh	TMR1H	Most Significant Byte of the 16-bit Timer TMR0							
10h	T1CON	T1GINV	TMR1GE	T1CKPS1	T1CKPS0	T1OSCEN	T1SYNC	TMR1CS	TMR1ON
11h	TMR2	Timer T2 Register							
12h	T2CON	-	TOUTPS3	TOUTPS2	TOUTPS1	TOUTPS0	TMR2ON	T2CKPS1	T2CKPS0
13h	SSPBUF	Synchronous Serial Port Receive Buffer/Transmit Register							
14h	SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0
15h	CCPR1L	Capture/ComparePWM Register 1 Low Byte (LSB)							
16h	CCPR1H	Capture/ComparePWM Register 1 High Byte (LSB)							
17h	CCP1CON	P1M1	P1M0	DC1B1	DC1B0	CCP1M3	CCP1M2	CCP1M1	CCP1M0
18h	RCSTA	SPEN	RX9	SREN	CREN	ADDEN	FERR	OERR	RX9D
19h	TXREG	EUSART Transmit Data Register							
1Ah	RCREG	EUSART Receive Data Register							
1Bh	CCPR2L	Capture/Compare PWM Register 1 Low Byte (LSB)							
1Ch	CCPR2H	Capture/Compare PWM Register 1 High Byte (LSB)							
1Dh	CCP2CON	-	-	DC2B1	DC2B0	CCP2M3	CCP2M2	CCP2M1	CCP2M0
1Eh	ADRESH	A/D Result Register High Byte							
1Fh	ADCON0	ADCS1	ADCS0	CHS3	CHS2	CHS1	CHS0	GO/DONE	ADON

SFRs bank 1

Address	Name	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
80h	INDF	Indirect Register							
81h	OPTION_REG	RBP	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0
82h	PCL	Least Significant Byte of Program Counter							
83h	STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C
84h	FSR	Indirect Data Memory Address Pointer							
85h	TRISA	TRISA7	TRISA6	TRISA5	TRISA4	TRISA3	TRISA2	TRISA1	TRISA0
86h	TRISB	TRISB7	TRISB6	TRISB5	TRISB4	TRISB3	TRISB2	TRISB1	TRISB0
87h	TRISC	TRISC7	TRISC6	TRISC5	TRISC4	TRISC3	TRISC2	TRISC1	TRISC0
88h	TRISD	TRISD7	TRISD6	TRISD5	TRISD4	TRISD3	TRISD2	TRISD1	TRISD0
89h	TRISE	-	-	-	-	TRISE3	TRISE2	TRISE1	TRISE0
8Ah	PCLATH	-	-	-	Upper 5 bits of the Program Counter				
8Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF
8Ch	PIE1	-	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE
8Dh	PIE2	OSFIE	C2IE	C1IE	EEIE	BCLIE	ULPWUE	-	CCP2IE
8Eh	PCON	-	-	ULPWUE	SBOREN	-	-	POR	BOR
8Fh	OSCCON	-	IRCF2	IRCF1	IRCF0	OSTS	HTS	LTS	SCS
90h	OSCTUNE	-	-	-	TUN4	TUN3	TUN2	TUN1	TUN0
91h	SSPCON2	GCEN	ACKSTAT	ACKDT	ACKEN	RCEN	PEN	RSEN	SEN
92h	PR2	Timer T2 Period Register							
93h	SSPAD	Synchronous Serial Port (I ² C mode) Address Register							
93h	SSPMASK	MSK7	MSK6	MSK5	MSK4	MSK3	MSK2	MSK1	MSK0
94h	SSPSTAT	SMP	CKE	D/A	P	S	R/W	UA	BF
95h	WPUB	WPUB7	WPUB6	WPUB5	WPUB4	WPUB3	WPUB2	WPUB1	WPUB0
96h	IOCB	IOCB7	IOCB6	IOCB5	IOCB4	IOCB3	IOCB2	IOCB1	IOCB0
97h	VRCON	VREN	VROE	VRR	VRSS	VR3	VR2	VR1	VR0
98h	TXSTA	CSRC	TX9	TXEN	SYNC	SEnDB	BRGH	TRMT	TX9D
99h	SPBRG	BRG7	BRG6	BRG5	BRG4	BRG3	BRG2	BRG1	BRG0
9Ah	SPBRGH	BRG15	BRG14	BRG13	BRG12	BRG11	BRG10	BRG9	BRG8
9Bh	PWM1CON	PRSEN	PDC6	PDC5	PDC4	PDC3	PDC2	PDC1	PDC0
9Ch	ECCPAS	ECCPASE	ECCPAS2	ECCPAS1	ECCPAS0	PSSAC1	PSSAC0	PSSBD1	PSSBD0
9Dh	PSTRCON	-	-	-	STRSYNC	STRD	STRC	STRB	STRA
9Eh	ADRESL	A/D Result Register Low Byte							
9Fh	ADCON1	ADFM	-	VCFG1	VCFG0	-	-	-	-

SFRs bank 2

Address	Name	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
100h	INDF	Indirect register							
101h	TMR0	Timer T0 Register							
102h	PCL	Least Significant Byte of the Program Counter							
103h	STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C
104h	FSR	Indirect Data Memory Address Pointer							
105h	WDTCON	-	-	-	WDTPS3	WDTPS2	WDTPS1	WDTPS0	SWDTEN
106h	PORTB	RB7	RB6	RB5	RB4	RB3	RB2	RB1	RB0
107h	CM1CON0	C1ON	C1OUT	C1OE	C1POL	-	C1R	C1CH1	C1CH0
108h	CM2CON0	C2ON	C2OUT	C2OE	C2POL	-	C2R	C2CH1	C2CH0
109h	CM2CON1	MC1OUT	MC2OUT	C1RSEL	C2RSEL	-	-	T1GSS	C2SYNC
10Ah	PCLATH	-	-	-	Upper 5 bits of the Program Counter				
10Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF
10Ch	EEDAT	EEDAT7	EEDAT6	EEDAT5	EEDAT4	EEDAT3	EEDAT2	EEDAT1	EEDAT0
10Dh	EEADR7	EEADR6	EEADR5	EEADR4	EEADR3	EEADR2	EEADR1	EEADR0	
10Eh	EEDATH	-	-	EEDATH5	EEDATH3	EEDATH2	EEDATH1	EEDATH0	
10Fh	EEADRH	-	-	-	EEADRH4	EEADRH3	EEADRH2	EEADRH1	EEADRH0

SFRs bank 3

Address	Name	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
180h	INDF	Indirect Register							
181h	OPTION_REG	RBP	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0
182h	PCL	Least Significant Byte of the Program Counter							
183h	STATUS	IRP	RP1	RP0	TO	PD	Z	DC	C
184h	FSR	Indirect Data Memory Address Pointer							
185h	SRCON	SR1	SR0	C1SEN	C2REN	PULSS	PULSR	-	FVREN
186h	TRISB	TRISB7	TRISB6	TRISB5	TRISB4	TRISB3	TRISB2	TRISB1	TRISB0
187h	BAUDCTL	ABDOVF	RCIDL	-	SCKP	BRG16	-	WUE	ABDEN
188h	ANSEL	ANS7	ANS6	ANS5	ANS4	ANS3	ANS2	ANS1	ANS0
189h	ANSELH	-	-	ANS13	ANS12	ANS11	ANS10	ANS9	ANS8
19Ah	PCLATH	-	-	-	Upper 5 bits of the Program Counter				
19Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF
19Ch	EECON1	EEPGD	-	-	-	WRERR	WREN	WR	RD
19Dh	EECON2	EEPROM Control Register 2							

STACK

A part of RAM used as stack consists of eight 13-bit registers. Before the microcontroller starts to execute a subroutine (CALL instruction) or when an interrupt occurs, the address of the first next instruction to execute is pushed onto the stack, i.e. one of its registers. Thanks to that the microcontroller knows from where to continue regular program execution upon a subroutine or an interrupt execution. This address is cleared after returning to the program because there is no need to save it any longer, and one location of the stack becomes automatically available for further use.

It is important to bear in mind that data is always circularly pushed onto the stack. It means that after the stack has been pushed eight times, the ninth push overwrites the value that was stored with the first push. The tenth push overwrites the second push and so on. Data overwritten in this way is not recoverable. In addition, the programmer cannot access these registers for write or read and there is no Status bit to indicate stack overflow or stack underflow conditions. For this reason, it is necessary to take special care of it during program writing.

Let's do it in mikroC...

```

1  /* When entering or exiting an assembly instruction in the program, the compiler
2  doesn't save data on the currently active RAM bank. It means that in this program
3  section, bank selection depends on the SFR registers in use. When switching back
4  to the program section written in C, the control bits RP0 and RP1 must return the
5  state they had before assembly language code execution. In this example, the problem
6  is solved by using the saveBank auxiliary variable which saves the state of
7  these two bits. */
8
9      saveBank = STATUS & 0b01100000; // Save the state of bits RP0 and RP1
10                                     // (bits 5 and 6 of the STATUS register)
11  <b>asm</b> {                          // Start of assembly sequence
12  ...
13  ...                                // Assembly code
14  ...
15  }                                  // End of assembly sequence
16  STATUS &= 0b10011111;              // Bits RP0 and RP1 return their original state
17  STATUS |= saveBank;
18  ...
19  ...

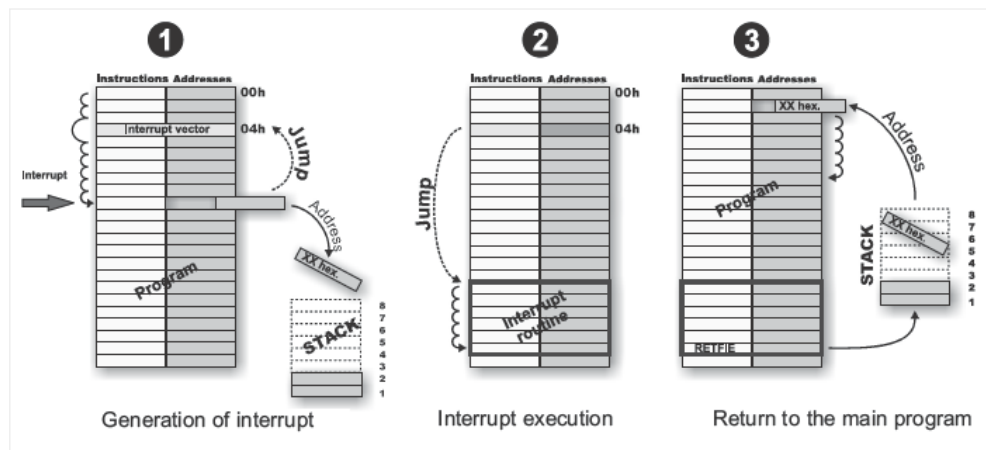
```

INTERRUPT SYSTEM

The first thing the microcontroller does when an interrupt request arrives is to execute the current instruction and then stops the regular program execution. As a result, the current program memory address is automatically pushed onto the stack and the default address (predefined by the manufacturer) is written to the program counter. The location from where the program proceeds with execution is called an interrupt vector. For the PIC16F887 microcontroller, this address is 0004h. As seen in figure below, the location containing the interrupt vector is passed over during regular program execution.

A part of the program to be executed when an interrupt request arrives is called an interrupt routine. Its first instruction is located at the interrupt vector. How long will it take to execute this subroutine and what it will be like depends on the skills of the programmer as well as on the interrupt source itself. Some of the microcontrollers have more interrupt vectors (every interrupt request has its vector), but in this case there is only one. Consequently, the first part of the interrupt routine consists in interrupt source detection.

Finally, when the interrupt source is recognized and the interrupt routine is executed, the microcontroller reaches the RETFIE instruction, pops the address from the stack and proceeds with program execution from where it left off.



mikroC recognizes an interrupt routine to be executed as the `void interrupt()` function. The body of that function, i.e. interrupt routine, should be written by the user.

```
1 void interrupt() { // Interrupt routine
2   cnt++;           // Interrupt causes variable cnt to be incremented by 1
3 }
```

In Short: How to Use SFRs

You have bought the microcontroller and have a good idea how to use it... There is a long list of SFRs and their bits. Each of them controls some process. All in all, it looks like a big control table with a lot of instruments and switches. Now you are concerned about whether you will manage to learn how to use them all? You will probably not, but don't worry, you don't have to! Such powerful microcontrollers are similar to supermarkets: they offer so many things at low prices and it is up to you to choose those you need. Therefore, select the field you are interested in and study only what you need to know. When you completely understand hardware operation, study SFRs which are in control of it (there are usually a few of them).

As all devices have a sort of control system, the microcontroller has its 'levers' which you have to be familiar with in order to be able to use it properly. Of course, we are talking about SFRs from which the process of programming begins and where it ends.



3.1 The PIC16F887 Basic Features by MikroElektronika is licensed under a Creative Commons Attribution 4.0 International License, except where otherwise noted.